Object Oriented Programming

Course Code: 25CSH-208

UNIT-2 CHAPTER -2.1

IO PACAKAGE

UNIT-2	IO PACAKGE AND MULTITHREADING	Contact Hours:15
IO package	Introduction to IO Package and Streams, Input streams, Output streams, Object serialization, De-serialization, Filter and pipe streams, IOFiles, Sample programs on IO files.	
Multi-threading	Thread Lifecycle, Multi-threading, Advantages and Issues of Multi-threading, Simple thread program, Thread-synchronization.	

FILES

- A **file** refers to a specific storage area on media where data can be stored (a hard disk, CD etc).
- **Data** in a file is stored as a sequence of bytes, but can be interpreted in different ways during **reading** and **writing**.
- If the **data in a file** is interpreted as **sequence of bytes**, the file is referred to as a **Binary file**.
- **Binary files** are created by using **byte streams**.
- If the **data in a file** is interpreted as **sequence of characters** , the file is referred as a **text file**.
- **Text files** are created using **character streams**.
- In java, files are represented by **java.io.File** class.

FILE CLASS

- The **java.io.File** class is a utility class used for working with files and directories.
- This class provides you the ability to list files and directories, obtain file status(read & write permissions), rename and delete files and directories, and perform other file operations.

- The **File class** is concerned with the storage of files, and the **Stream classes** are concerned with the content of files.
- A **file or directory's path** specifies its location on a disk.
- A path can be either **absolute or relative**.
- An **absolute path** contains all the directories, starting with the root directory, that lead to a specific file or directory.
- For example, C:\Users\91842\JAVA_PROGRAM\UNIT 2\ FileIOStreamDemo.java, is the absolute path of FileIOStreamDemo.java.
- A **relative path** specifies the path relative to the current Directory- the directory in which the program is running.

FILE CLASS METHODS

Method	Description
`boolean canRead()`	Returns `true` if a file is readable by the current application; `false` otherwise.
`boolean canWrite()`	Returns `true` if a file is writable by the current application; `false` otherwise.
`boolean exists()`	Returns `true` if the file or directory represented by the `File` object exists; `false` otherwise.
`boolean isFile()`	Returns `true` if the name specified as the argument to the `File` constructor is a file; `false` otherwise.
`boolean isDirectory()`	Returns `true` if the name specified as the argument to the `File` constructor is a directory; `false` otherwise.
`boolean isAbsolute()`	Returns `true` if the arguments specified to the `File` constructor indicate an absolute path to a file or directory; `false` otherwise.

Method	Description
`String getAbsolutePath()`	Returns a `String` with the absolute path of the file or directory.
`String getName()`	Returns a `String` with the name of the file or directory.
`String getPath()`	Returns a `String` with the path of the file or directory.
`String getParent()`	Returns a `String` with the parent directory of the file or directory (i.e., the directory in which the file or directory is located).
`long length()`	Returns the length of the file, in bytes. If the `File` object represents a directory, an unspecified value is returned.
`long lastModified()`	Returns a platform-dependent representation of the time at which the file or directory was last modified. The value returned is useful only for comparison with other values returned by this method.
`String[] list()`	Returns an array of `String`s representing a directory's contents. Returns `null` if the `File` object does not represent a directory.

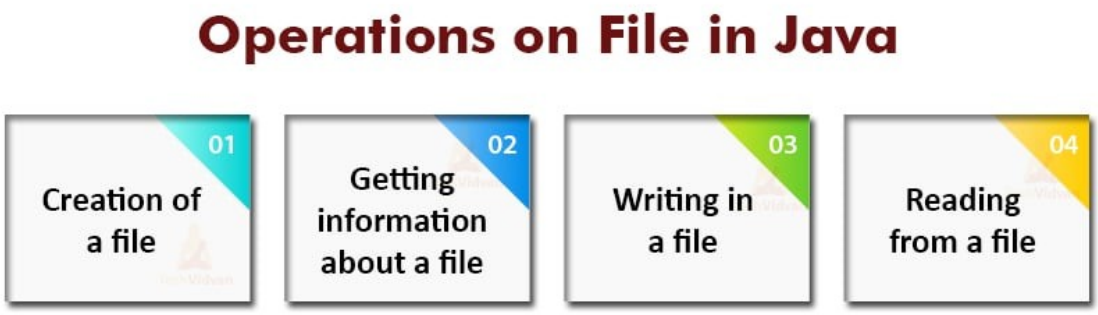
Example 1: Using the File Class and its methods

```
import java.io.*;
public class FileDemo {
   public static void main (String []args)
   {
      File file = new File("Rectangle1.java");
      System.out.println("File exists....>"+file.exists());
      System.out.println("Length of a file....>"+file.length());
      System.out.println("Can Read file....>"+file.canRead());
      System.out.println("Can Write to File....>"+file.canWrite());
      System.out.println("Whether file is a directory....>"+file.isDirectory());
      System.out.println("Whether File is file ....>"+file.isFile());
      System.out.println("File name is....>"+file.getName());
      System.out.println("Files absolute path is...>"+file.getAbsolutePath());
      System.out.println("File path is....>"+file.getPath());
      System.out.println("Files parent directory....>"+file.getParent());
   }
}
```

OUTPUT:

```
PS C:\Users\91842\JAVA_PROGRAM\UNIT 2> javac FileDemo.java
PS C:\Users\91842\JAVA_PROGRAM\UNIT 2> java FileDemo
File exists....>false
Length of a file....>0
Can Read file....>false
Can Write to File....>false
Whether file is a directory....>false
Whether File is file ....>false
File name is....>Rectangle1.java
Files absolute path is...>C:\Users\91842\JAVA_PROGRAM\UNIT 2\Rectangle1.java
File path is....>Rectangle1.java
Files parent directory....>null
```

OPERATIONS ON FILES IN JAVA



1. **Creation of a File:** Java provides a method createNewFile() that provides a functionality to create a new file on a specified directory. The createNewFile() method returns a Boolean value based on the creation of the file.

2. **Getting information about a file:** To get the information about the file in Java, we extract the length of the file, the path to the file, and the permissions granted to the file(read and write) with the help of different methods.

3. **Writing in a File:** In Java, there is a class called FileWriter that is responsible for writing the content into the file. If we want to write the content in the file, we need to create an object of the FileWriter class and then provide the absolute path of the file to that object.

4. **Reading from a File:** To read the content or data from the file, we need two classes File and Scanner. The Scanner class reads the contents from the file.The File object acts as a bridge between the Scanner and the content of the file. We pass the object of the file class to the Scanner class.

1.FileOutputStream Class:

- The FileOutputStream class in Java is used to write data to a file in the form of Raw bytes. That means you can use it not only for text files, but also for **binary files** (like images, PDFs, or audio).
- For writing character data, We should use FileWriter instead.

- **Byte-Oriented Stream:** Writes data byte by byte.
- **Direct Access:** Writes data directly to the disk without buffering.
- **Platform Independent:** Works across all operating systems.

- **Declaration of FileOutputStream**

public class FileOutputStream extends OutputStream

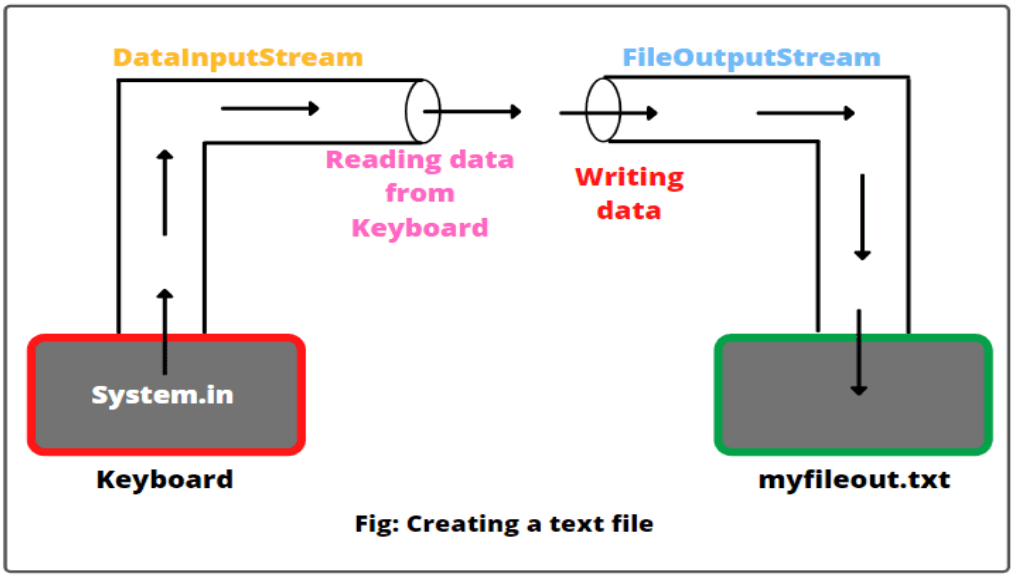


Fig: Creating a text file

Methods of FileInputStream Class

Method	Description
void write(int b)	Writes the specified byte to this file output stream.
void write(byte[] b)	Writes `b.length` bytes from the specified byte array to this file output stream.
void write(byte[] b, int off, int len)	Writes `len` bytes from the specified byte array starting at offset `off` to this file output stream.
FileChannel getChannel()	Returns the unique FileChannel object associated with this file output stream.
FileDescriptor getFD()	Returns the file descriptor associated with this stream.
protected void finalize()	Cleans up the connection to the file, and ensures that the `close` method of this file output stream is called when there are no more references to this stream.
void close()	Closes this file output stream and releases any system resources associated with this stream.

Example2:Write a single byte to a file using FileOutputStream class

```
import java.io.FileOutputStream;
public class FileOutputStreamExample1 {
   public static void main(String args[]) {
      try {
         FileOutputStream fout = new FileOutputStream("D:\\testout.txt");
```

```
            fout.write(65);
            fout.close();
            System.out.println("success...");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT:

```
success...
```

Example3:Write a string to a file using FileOutputStream class

```
import java.io.FileOutputStream;
public class FileOutputStreamExample {
    public static void main(String[] args) {
        try {
            // Create FileOutputStream object (file will be created if not exists)
            FileOutputStream fout = new FileOutputStream("output.txt");
            String data = "Hello Students! This is FileOutputStream example.";
            // Convert string into byte array
            byte b[] = data.getBytes();
            // Write data into file
            fout.write(b);
            // Close the stream
            fout.close();
            System.out.println("Data successfully written to file.");
        } catch (Exception e) {
            System.out.println(e);
        }
```

```
    }
}
```

Output:

Data successfully written to file.

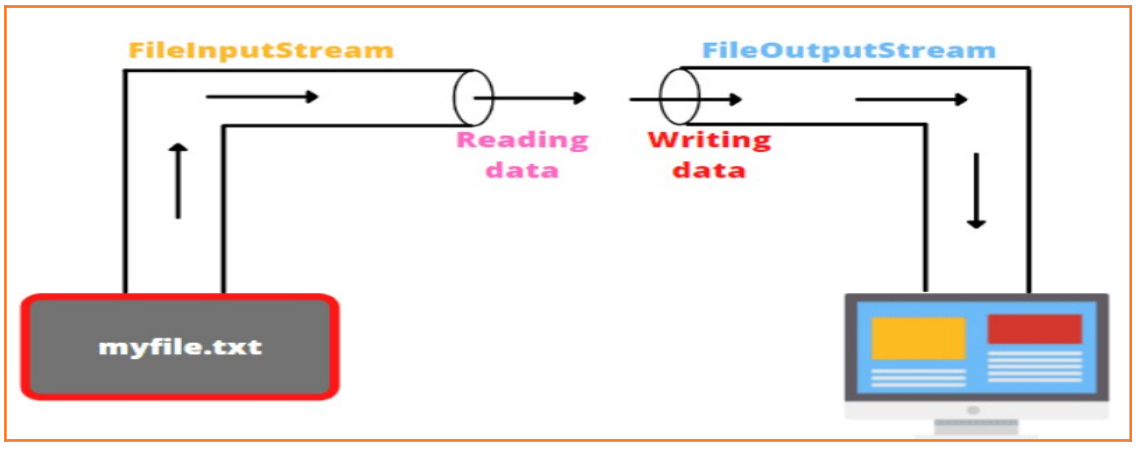
2. FileInputStream Class

- The FileInputStream class **creates an InputStream that** you can use to read bytes from a file.

  FileInputStream(String *filepath)* throw a FileNotFoundException

- ***filepath*** specifies the name of the file you want to open.
- **FileNotFoundException** is thrown- If the file does not exist.
- **FileNotFoundException** is a subclass of **IOException**.
- To read from a file, you can use **read( )**.
- **Eg::**
- FileInputStream f0 = new FileInputStream("/autoexec.bat")
  File f = new File("/autoexec.bat");
  FileInputStream f1 = new FileInputStream(f);

- FileInputStream overrides six of the methods in the abstract class InputStream.
- The mark( ) and reset( ) methods are not overridden, and any attempt to use reset( ) on a FileInputStream will generate an IOException



Example 4: Read a single character from a Java FileInputStream

import java.io.FileInputStream;

```
public class DataStreamExample1 {
   public static void main(String[] args) {
      try {
         FileInputStream fin = new FileInputStream("C:\\Users\\91842\\JAVA_PROGRAM\\
UNIT 2\\mydata2.txt");
         int i;
         i = fin.read();          // Read only ONE character
         if (i != -1) {
            System.out.print((char) i);
         } else {
            System.out.println("File is empty");
         }
         fin.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Output:

W

Note: Before running the code, a text file named **"testout.txt"** is required to be created. In this file, we are having the following content:

WELCOME TO UNIT 2 CHAPTER IO STREAMS

After executing the above program, you will get a single character from the file which is 87 (in byte form). To see the text, you need to convert it into character.

Example 5: Read all characters from a Java FileInputStream

```
import java.io.FileInputStream;
public class DataStreamExample {
   public static void main(String[] args) {
      try {
```

```
        // Open file (use correct path format)
        FileInputStream file1 = new FileInputStream("C:\\Users\\91842\\JAVA_PROGRAM\\
UNIT 2\\mydata2.txt");
        int i;
        // Read file character by character
        while ((i = file1.read()) != -1) {
           System.out.print((char) i);
        }
        file1.close(); // Close the file
      } catch (Exception e) {
        System.out.println(e);
      }
   }
}
```

OUTPUT:

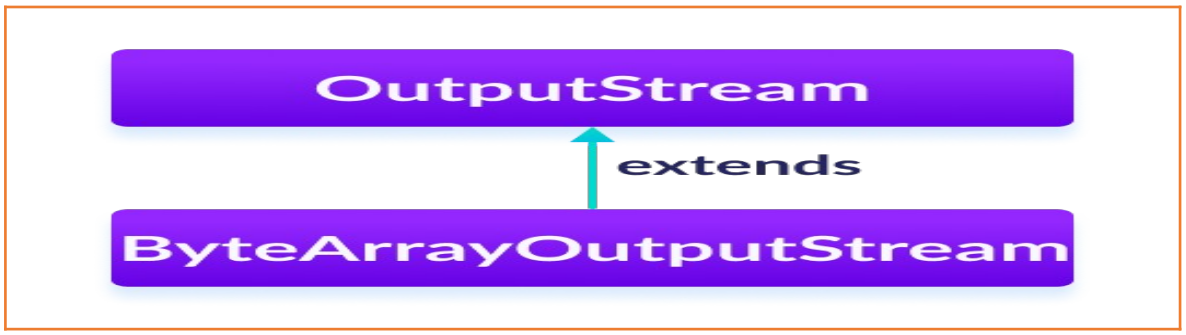
WELCOME TO UNIT 2 CHAPTER IO STREAMS

3. ByteArrayOutputStream

- **ByteArrayOutputStream** is an implementation of an **output stream** that uses a byte array as the destination.
- The ByteArrayOutputStream class writes data into an internal byte array instead of directly writing it to a file or device. The stored data can later be written to multiple output streams.
- This class maintains a copy of data in memory and forwards it to multiple streams when required. The internal buffer automatically grows as more data is written.
- ByteArrayOutputStream Class Declaration

```
Public class ByteArrayOutputStream extends OutputStream
```

- **Constructors:**
  ByteArrayOutputStream( ) // a buffer of 32 bytes is created
  ByteArrayOutputStream(int numBytes)



ByteArrayOutputStream class methods

Method	Description
int size()	It is used to returns the current size of a buffer.
byte[] toByteArray()	It is used to create a newly allocated byte array.
String toString()	It is used for converting the content into a string decoding bytes using a platform default character set.
String toString(String charsetName)	It is used for converting the content into a string decoding bytes using a specified charsetName.
void write(int b)	It is used for writing the byte specified to the byte array output stream.
void write(byte[] b, int off, int len	It is used for writing len bytes from specified byte array starting from the offset off to the byte array output stream.
void writeTo(OutputStream out)	It is used for writing the complete content of a byte array output stream to the specified output stream.
void reset()	It is used to reset the count field of a byte array output stream to zero value.
void close()	It is used to close the ByteArrayOutputStream.

Example6:Program using ByteArrayOutputStream

```
import java.io.ByteArrayOutputStream;

public class ByteArrayOutputStreamDemo {

   public static void main(String[] args) {

      try {

         ByteArrayOutputStream bout = new ByteArrayOutputStream();

         String data = "Hello BTech 2nd year Students";

         // Convert string to byte array
```

```
            byte[] b = data.getBytes();
            // Write data into stream
            bout.write(b);
// Convert back to string
            byte[] result = bout.toByteArray();
System.out.println("Data in ByteArrayOutputStream:");
            System.out.println(new String(result))
            bout.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Output:

Data in ByteArrayOutputStream:

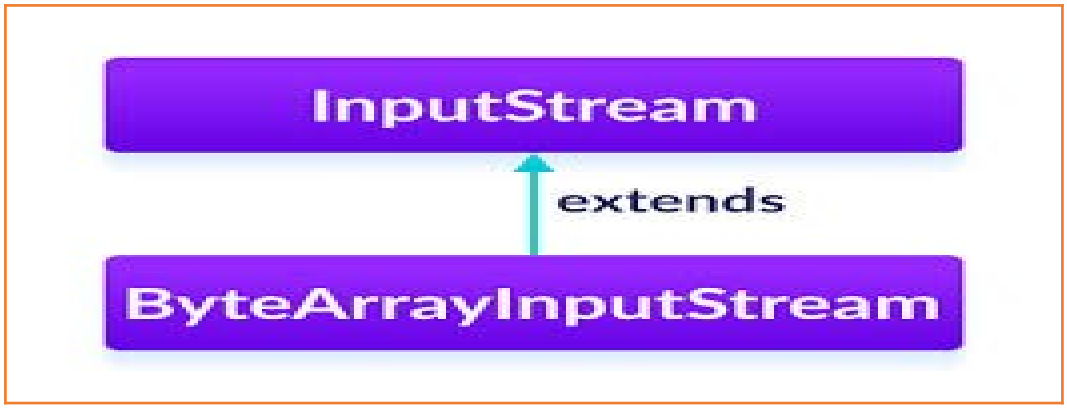
Hello BTech 2nd year Students

4. ByteArrayInputStream

- ByteArrayInputStream is an implementation of an input stream that uses a byte array as the source.

ByteArrayInputStream(byte array[ ])
ByteArrayInputStream(byte array[ ], int start, int numBytes)

- Java ByteArrayInputStream class contains an internal buffer which is used to read byte array as stream. In this stream, the data is read from a byte array.
- The buffer of ByteArrayInputStream automatically grows according to data.



ByteArrayInputStream class methods

Methods	Description
int available()	It is used to return the number of remaining bytes that can be read from the input stream.
int read()	It is used to read the next byte of data from the input stream.
int read(byte[] ary, int off, int len)	It is used to read up to len bytes of data from an array of bytes in the input stream.
boolean markSupported()	It is used to test the input stream for mark and reset method.
long skip(long x)	It is used to skip the x bytes of input from the input stream.
void mark(int readAheadLimit)	It is used to set the current marked position in the stream.
void reset()	It is used to reset the buffer of a byte array.
void close()	It is used for closing a ByteArrayInputStream.

Example7: Program using ByteArrayInputStream

```
import java.io.ByteArrayInputStream;
public class ByteArrayInputStreamDemo {
    public static void main(String[] args) {
        try {
            String data = "Welcome Students in Java Class";
            // Convert string to byte array
            byte[] b = data.getBytes();
            // Create input stream from byte array
```

```
        ByteArrayInputStream bin = new ByteArrayInputStream(b);
        int i;
        System.out.println("Reading data using ByteArrayInputStream:");
 // Read data
        while ((i = bin.read()) != -1) {
           System.out.print((char) i);
        }
        bin.close();
      } catch (Exception e) {
        System.out.println(e);
      }
   }
}
```

OUTPUT:

Reading data using ByteArrayInputStream:

Welcome Students in Java Class

5. ObjectOutputStream

- Java also supports writing and reading objects to a stream(such as files).
- The process of writing an object to a file is called **Object serialization.**
- In Java, **serialization** occurs automatically.
- Objects can be converted into a sequence of bytes or serialized by using the **java.io.ObjectOutputStream** class.
- The **ObjectOutputStream** class extends **OutputStream** and implements the **ObjectOutput** interface.
- The ObjectOutput interface extends the DataOutput interface, which declares methods for writing data for primitive types.
- In addition, ObjectOutput adds writeObject() method which writes objects to a stream. It takes the following form:

void writeObject(Object obj) throws IOException

Here, obj is the object to be wriiten to the output stream.

- The Object output stream can be created using the following constructor

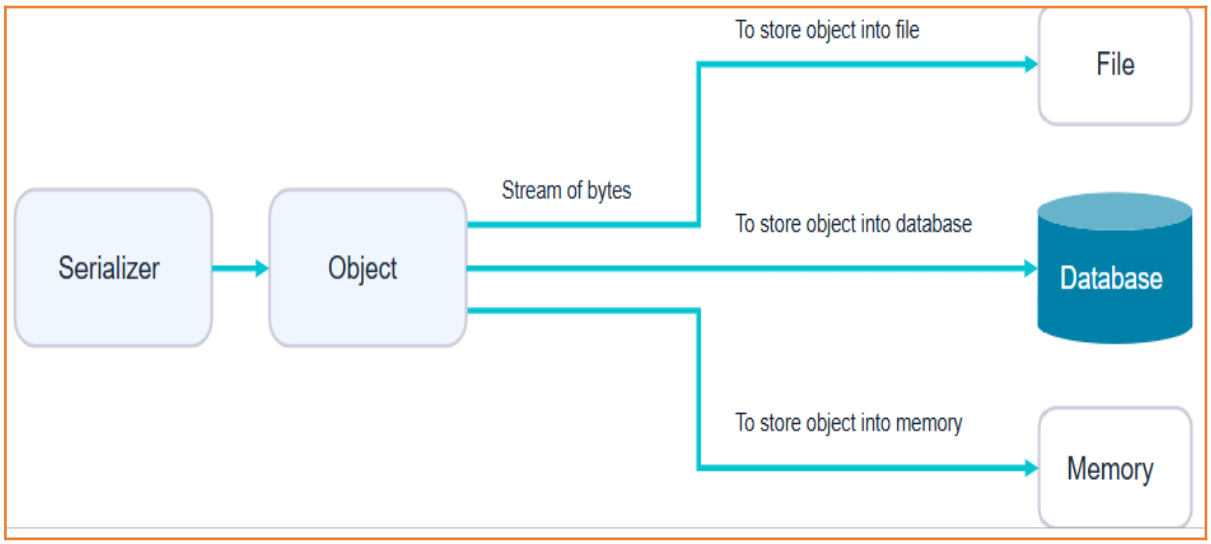
Object OutputStream(OutputStream outStream)

Here, outstream is the output stream to which serialized objects will be written. Constructor throws an IOexception if the file cannot be opened.

How to Write to an ObjectOutputStream

```
FileOutputStream out = new FileOutputStream("theTime");
ObjectOutputStream s = new ObjectOutputStream(out);
s.writeObject("Today");
s.writeObject(new Date());
s.flush();
```

Object serialization



- **transient keyword** is used in Object Serialization.
- By default, when you serialize an object, all its fields are serialized except for static variables. When you construct this object back from its persistent state, you will get the values of all the fields that are serialized(except static variables).
- If you do not want to store the value of a particular non-static field, then you can declare this field as transient.
- This keyword is used only with a variable declaration.

6. ObjectInputStream

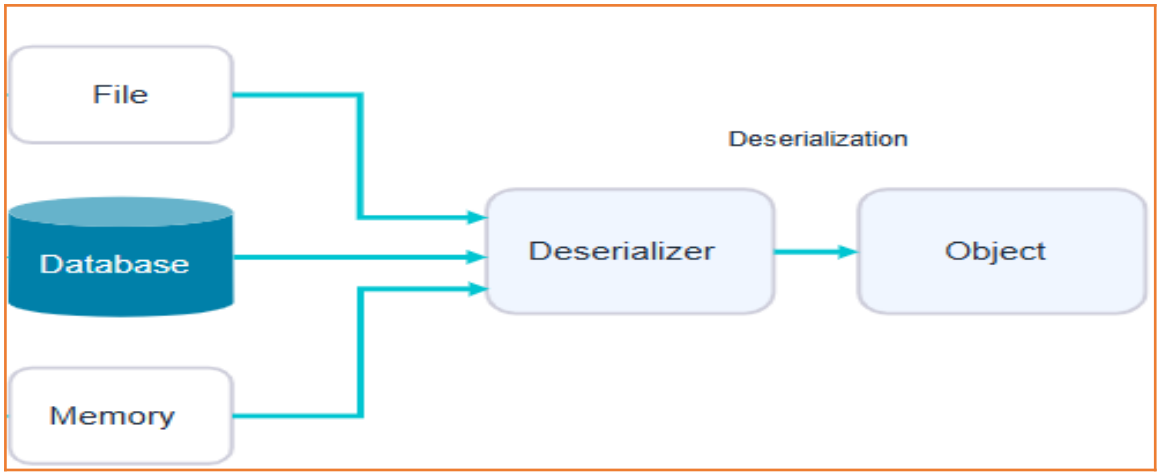
- The process of reading the object back from a file is called **Object deserialization**.
- Objects can be converted from bytes into a structured object or deserialized by using the **java.io.ObjectInputStream** class.
- The ObjectInputStream extends **InputStream** and implements the **ObjectInput** and ObjectStreamConstants interface.
- The ObjectInput interface extends the DataInput interface, which declares methods for reading data of the primitive types.
- In addition, ObjectInput adds a readObject() method which reads object from a stream. It takes the following form:
- void readObject(Object obj) throws IOException, ClassNotFoundException

➢ The Object input stream can be created using the following constructor

Object InputStream(InputStream inStream)

Here, instream is the input stream from which objects will be read. The constructor throws an IOException if the file cannot be opened.

Object Deserialization



➢ After a serialized **object is written to a file, it can be read from the file** and deserialized (that is we can recreate the object in memory)

- The process **of Serialization and DeSerialization is JVM independent**. That is, an object can be serialized on one platform and deserialized on an entirely different platform.
- **Classes ObjectInputStream and ObjectOutputStream** are used for serialization & deserialization.
- **How to Read from an ObjectInputStream**

- FileInputStream in = new FileInputStream("theTime");
- ObjectInputStream s = new ObjectInputStream(in);
- String today = (String) s.readObject();
- Date date = (Date) s.readObject();

Example8: To implement object serialization & object deserialization

```
import java.io.*;
// Employee class implementing Serializable
class Employee implements Serializable {
   int empid;
   String ename;
   double salary;
   Employee(int id, String name, double sal) {
      empid = id;
      ename = name;
      salary = sal;
   }
 void showInformation() {
      System.out.println("Empid -- " + empid);
      System.out.println("Ename -- " + ename);
      System.out.println("Salary -- " + salary);
      System.out.println();
```

```
    }
}
// Main class
class ObjectIOStreamDemo {
    public static void main(String[] args) {
        Employee empData[] = {
            new Employee(101, "Aman", 15000),
            new Employee(102, "Sanju", 25000),
            new Employee(103, "Vikram", 16000)
        };
        // Writing objects to file
        try {
            FileOutputStream fout = new FileOutputStream("employee.dat");
            ObjectOutputStream objOut = new ObjectOutputStream(fout);
            for (int i = 0; i < empData.length; i++) {
                objOut.writeObject(empData[i]);
            }
            objOut.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
 // Reading objects from file
        try {
            FileInputStream fin = new FileInputStream("employee.dat");
            ObjectInputStream objIn = new ObjectInputStream(fin);
            Employee emp;
```

```
            emp = (Employee) objIn.readObject();
            emp.showInformation();
            emp = (Employee) objIn.readObject();
            emp.showInformation();
            emp = (Employee) objIn.readObject();
            emp.showInformation();
  objIn.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Empid -- 101
Ename -- Aman
Salary -- 15000.0
Empid -- 102
Ename -- Sanju
Salary -- 25000.0
Empid -- 103
Ename -- Vikram
Salary -- 16000.0
```

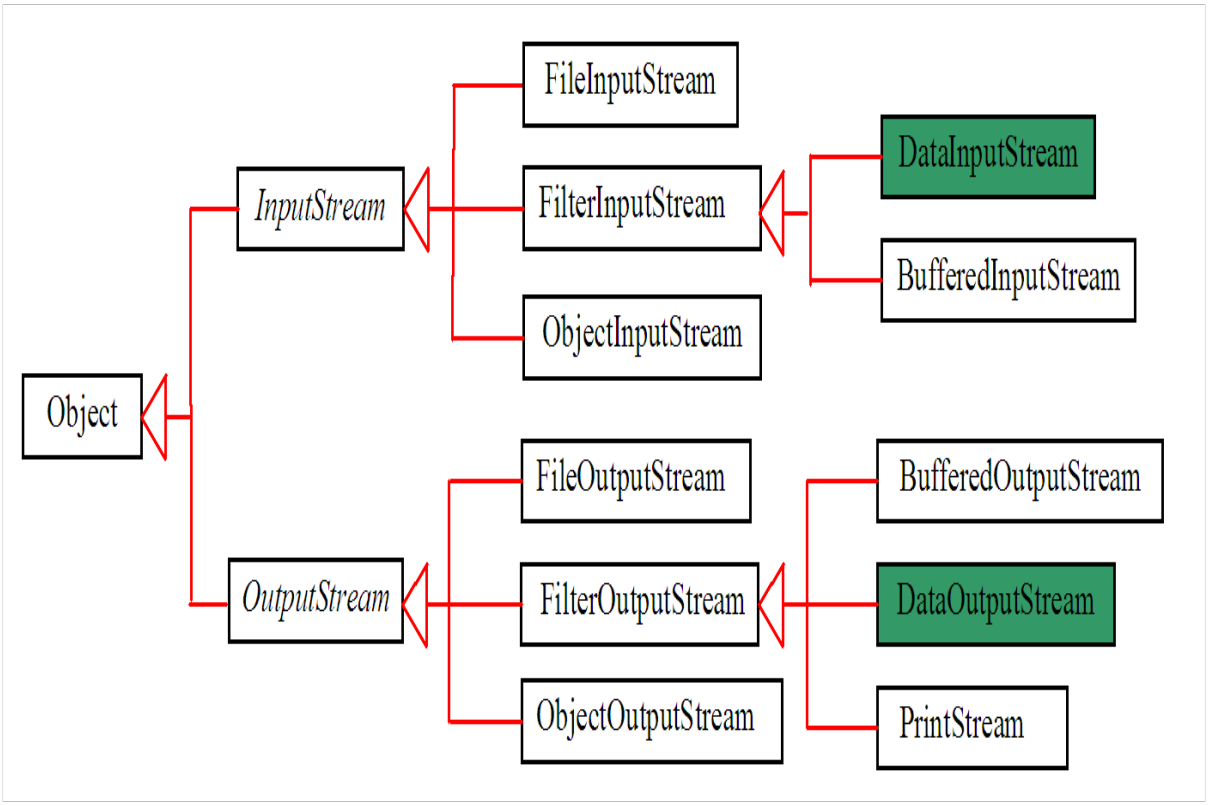
7.FilterStream classes

- What you do when you need to read integers, write floating point values, and read or write some other non-byte value from/to a file?
- The answer is to use **Filter Streams.**

- A **filter stream** is a stream that filters bytes (i.e modify or manipulates bytes) as is being read from or written to the stream.
- **A** Filter Stream is constructed on another stream.
- This means that the filter input stream do not read from input device such as files or sockets; rather, they read from preexisting input streams such as FileInputStream.
- Similarly, filter output stream do not write to output device but write to preexisting output streams such as FileOutputStream.

Filter streams are use for translation, buffering, encryption, compression and much more

7.1 FilterInputStream & FilterOutputStream

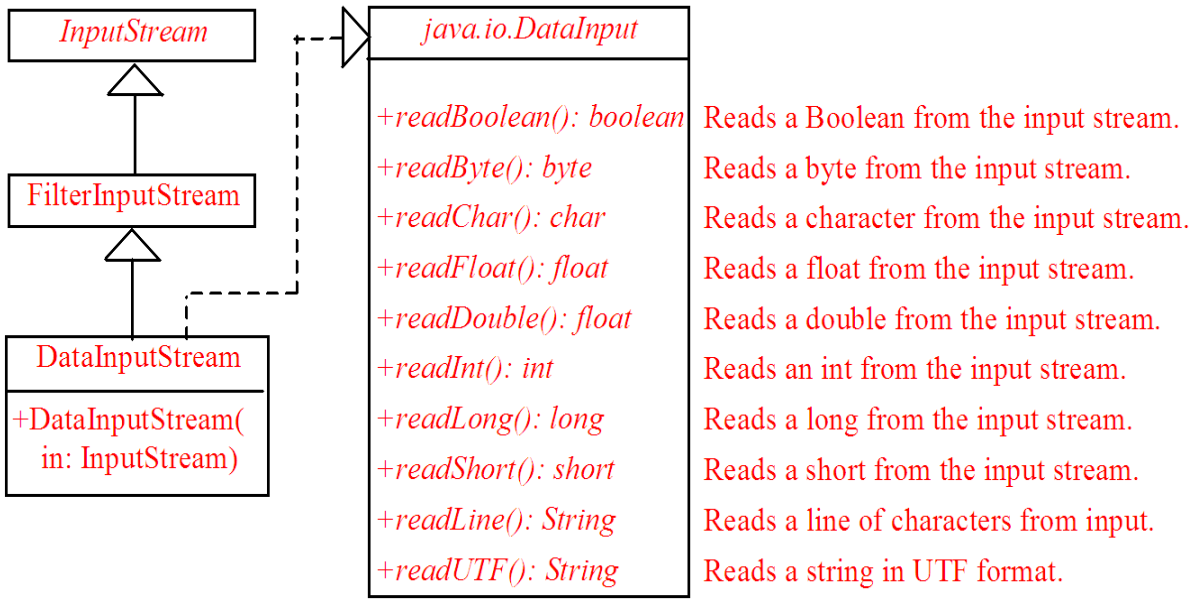


- Java.io.FilterInputStream and java.io.FilterOutputStream are the base classes for filtering data.
- These classes have subclasses that handle specific type of filtering.
- Some of these subclasses include:

- DataInputStream and DataOutputStream
- BufferedInputStream and BufferedOutputStream
- PushBackInputStream

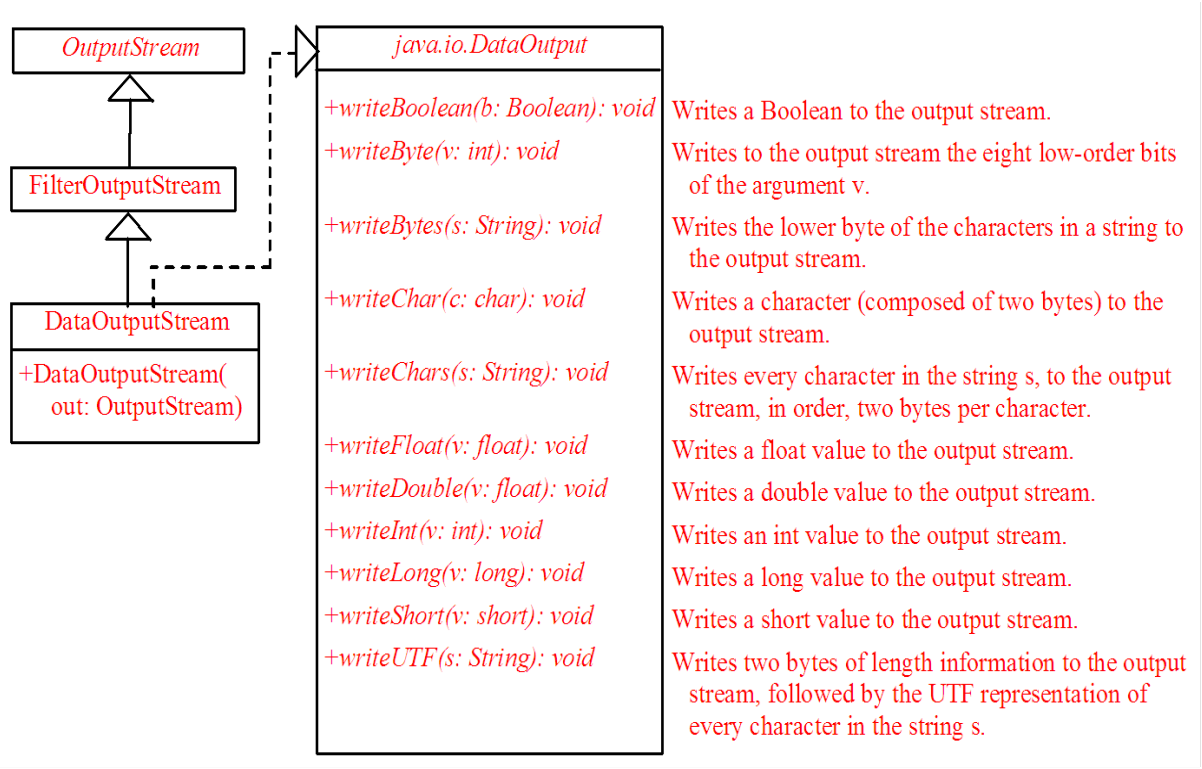
7.1.1DataInputStream

- DataInputStream reads bytes from the stream and converts them into appropriate primitive type values or strings.
- DataInputStream extends FilterInputStream and implements the DataInput interface



7.1.2 DataOutputStream

- DataOutputStream converts primitive type values or strings into bytes and output the bytes to the stream.
- DataOutputStream extends FilterOutputStream and implements the DataOutput interface.



Example9:Readin/ Writing Primitive Types: DataInputStream & DataOutputStream

- The following steps need to be followed to read or write primitive types from the file.
- Create the underlying file stream. For input, create an instance of FileInputStream, and for output, create an instance of FileOutputStream.
- Create an appropriate data stream by passing an instance of underlying File stream. For Input, create the DataInputStream by passing an instance of FileInputStream. For output, create the DataOutputStream by passing an instance of FileOutputStream.
- Perform all I/O operations through the data stream.
- Close the data stream. As a result, the underlying file stream is closed automatically.

```
import java.io.*;

class DataIOStreamDemo {

   public static void main(String[] args) {

      // Writing data to file
```

```
        try {
            // Create FileOutputStream
            FileOutputStream fout = new FileOutputStream("studentinfo.dat");
            // Create DataOutputStream
            DataOutputStream dout = new DataOutputStream(fout);
            // Write student data
            dout.writeInt(101);
            dout.writeUTF("Aman Verma");
            dout.writeDouble(70.5);
            dout.writeLong(2565879);
            dout.close(); // close output stream
        }
        catch (IOException e) {
            e.printStackTrace();
        }
    // Reading data from file
        try {
            // Create FileInputStream
            FileInputStream fin = new FileInputStream("studentinfo.dat");
            // Create DataInputStream
            DataInputStream din = new DataInputStream(fin);
            // Read student data
            System.out.println("Roll Number -- " + din.readInt());
            System.out.println("Name -- " + din.readUTF());
            System.out.println("Percentage -- " + din.readDouble());
            System.out.println("Phone Number -- " + din.readLong());
            din.close(); // close input stream
        }
        catch (IOException e) {
```

```
            e.printStackTrace();
        }
    }
}
```

OUTPUT:

Roll Number -- 101

Name -- Aman Verma

Percentage -- 70.5

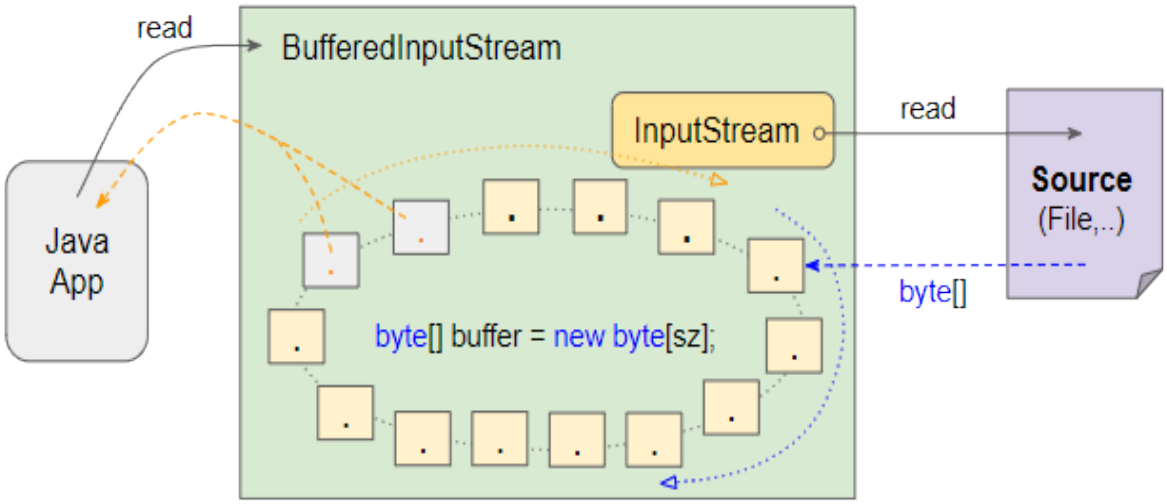
Phone Number – 2565879

Explanation:

- In this , we create **FileOutputStream** object "**fout**".
- Then we use **DataOutputStream** constructor to create an object, **dout**, to wrap the **FileOutputStream** object **fout**.
- Then, using the methods of the **DataOutputStream**, we write student data in order *roll number*, *name*, *percentage of marks*, *and phone number* to the file **studentinfo.dat**, and then we close the output stream
- To read the student information from the **studentinfo.dat**, a **FileInputStream** object **fin** is created, and using **the DataInputStream** constructor, we create an object **din** to wrap the **FileInputStream** object **fin**.
- Then read the data back from the file and display it on the console and finally close the input stream.

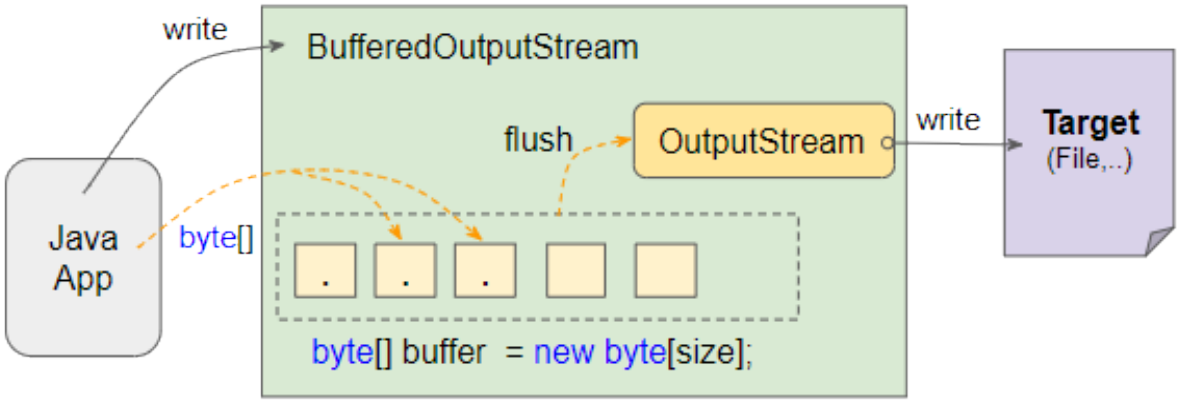
7.1.3 BufferedInputStream

- The BufferedInputStream class of the java.io package is used with other input streams to read the data (in bytes) more efficiently.
- It extends the InputStream abstract class.
- The BufferedInputStream maintains an internal **buffer of 8192 bytes**.
- During the read operation in BufferedInputStream, a chunk of bytes is read from the disk and stored in the internal buffer. And from the internal buffer bytes are read individually.
- Hence, the number of communication to the disk is reduced. This is why reading bytes is faster using the BufferedInputStream.



7.1.4 BufferedOutputStream

- The BufferedOutputStream class of the java.io package is used with other output streams to write the data (in bytes) more efficiently.
- It extends the OutputStream abstract class.
- The BufferedOutputStream maintains an internal **buffer of 8192 bytes**.
- During the write operation, the bytes are written to the internal buffer instead of the disk. Once the buffer is filled or the stream is closed, the whole buffer is written to the disk.
- Hence, the number of communication to the disk is reduced. This is why writing bytes is faster using BufferedOutputStream.



Example 10: Buffered Streams: BufferedInputStream & BufferedOutputStream

- The following steps need to be followed in order to buffer the file streams

- Create the underlying file stream. For input, create an instance of FileInputStream, and for output, create an instance of FileOutputStream.
- Create an appropriate buffered stream by passing an instance of underlying file stream.
- Perform all input/ouput operations through the buffered stream.
- Close the buffered stream. It automatically causes the underlying file stream to be closed.
- In next slide , we consider a program that writes the square of first 10 natural numbers to a file “mydata.dat” and then reads it backe and display it on the console

```
import java.io.*;
class BufferIOStreamDemo {
   public static void main(String[] args) {
      int ch;
      // Writing data to file
      try {
         // Create file output stream
         FileOutputStream fout = new FileOutputStream("mydata.dat");
         // Create buffered output stream
         BufferedOutputStream bout = new BufferedOutputStream(fout);
         // Write squares of first 10 natural numbers
         for (int i = 1; i <= 10; i++) {
            bout.write(i * i);
         }
         bout.close(); // close buffered output stream
      }
      catch (IOException e) {
         e.printStackTrace();
      }
// Reading data from file
      try {
         // Create file input stream
```

```
            FileInputStream fin = new FileInputStream("mydata.dat");
            // Create buffered input stream
            BufferedInputStream bin = new BufferedInputStream(fin);
            // Read data until EOF
            while ((ch = bin.read()) != -1) {
               System.out.println(ch);
            }
            bin.close(); // close buffered input stream
         }
         catch (IOException e) {
            e.printStackTrace();
         }
      }
}
```

Output:

1

4

9

16

25

36

49

64

81

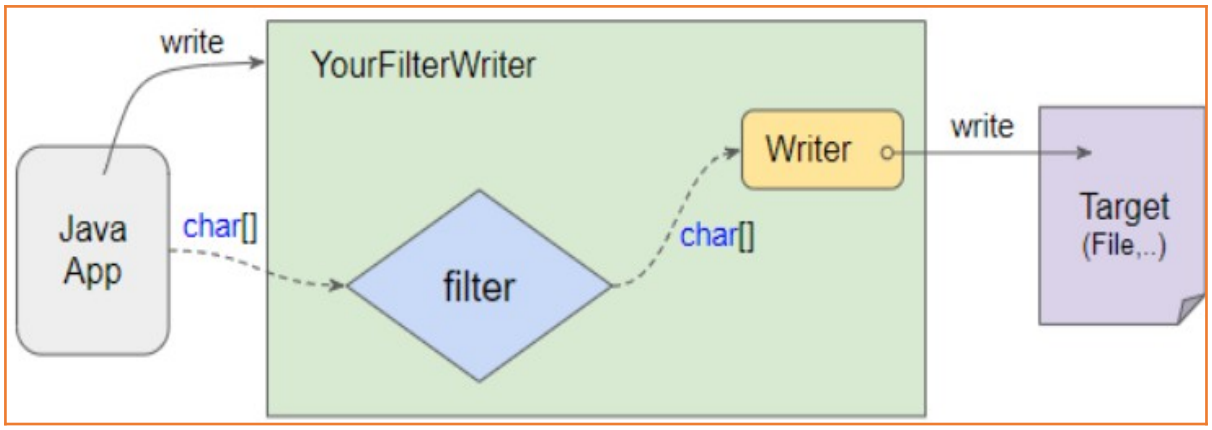
100

1.2.1.FilterWriter

- The Java FilterWriter class is an abstract class which is used to write filtered character streams.
- The sub class of the FilterWriter should override some of its methods and it may provide additional methods and fields also.

- Package: java.io
- Superclass: Writer
- Implements: Closeable, Flushable, Appendable
- Purpose: To provide a base for creating writers that filter data.
- Constructor

protected FilterWriter(Writer out)



Example11: using FilterWriter class

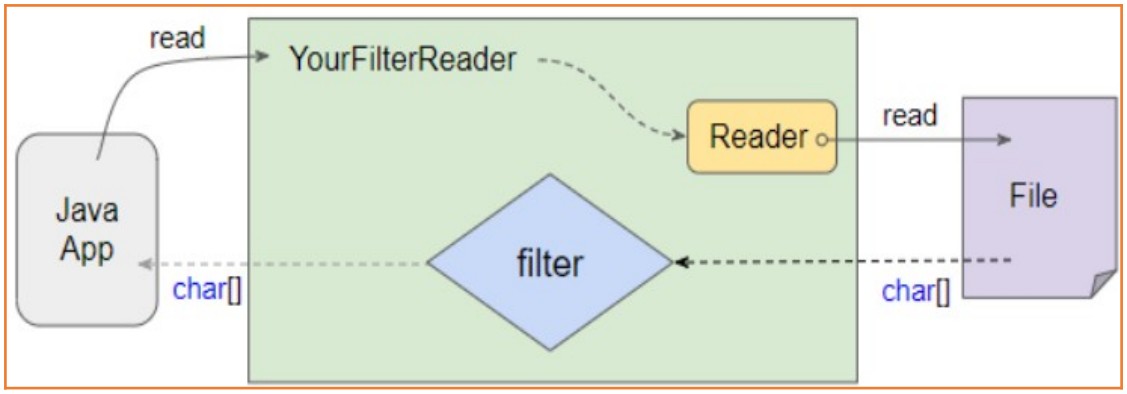
```
import java.io.*;
class FilterWriterDemo {
    public static void main(String[] args) throws Exception {
        FilterWriter fr = null;
        Writer wr = null;
        // Creating a StringWriter object
        wr = new StringWriter();
        // Creating an anonymous FilterWriter
        fr = new FilterWriter(wr) {};
        String str = "navjot";
        char c[] = {'n', 'a', 'v', 'j'};
        // Writing full string
        fr.write(str);
```

```
 // Writing part of string (offset 0, length 4 → "navj")
        fr.write(str, 0, 4);
        // Writing character array
        fr.write(c);
// Writing part of character array (offset 1, length 2 → "av")
        fr.write(c, 1, 2);
        // Flush and close
        fr.flush();
        fr.close();
        // Display output
        System.out.println("Output: " + wr.toString());
    }
}
```

```
PS C:\Users\91842\JAVA_PROGRAM\UNIT 2> javac FilterWriterDemo.java
PS C:\Users\91842\JAVA_PROGRAM\UNIT 2> java FilterWriterDemo
Output: navjotnavjnavjav
```

1.2.2 FilterReader Class

- The **Java.io.FilterReader class is for reading filtered character streams**.
- The class itself **provides default methods that pass all requests** to the contained stream.
- The **Subclasses of FilterReader should override some of these methods** and may also provide additional methods and fields.



Example 12: using FilterReader class

```
import java.io.*;
class FilterReaderDemo {
   public static void main(String[] args) throws IOException {
      // Create reader from string
      Reader r = new StringReader("navjot");
      // Wrap with FilterReader
      FilterReader fr = new FilterReader(r) {};
      int ch;
      System.out.print("Output: ");
      // Read character by character
      while ((ch = fr.read()) != -1) {
         System.out.print((char) ch);
      }
      fr.close();
   }
}
```

Output: navjot

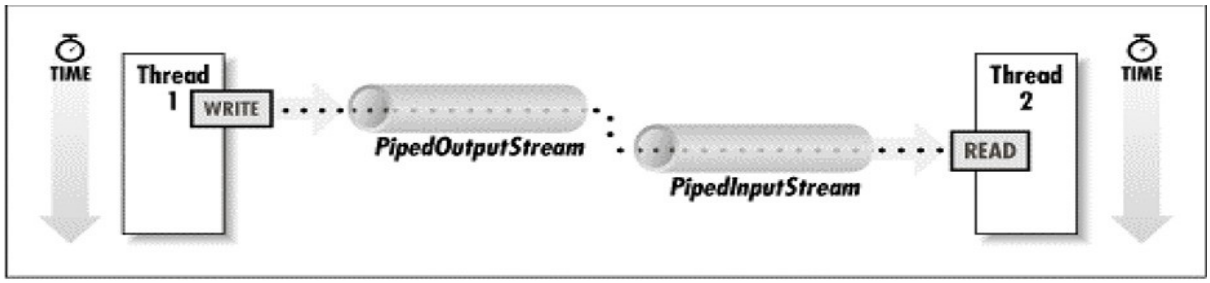
8. Java PipedStream Class

- **Pipes** are used as a communication channel between **threads to transfer data in a producer-consumer scenario.**
- Provide **unidirectional flow of data**, where one thread writes data into a pipe, and another thread reads from it.

- Useful for **thread-based communication**
- Classes to implement pipes in java.io
    - **PipedInputStream**
    - **PipedOutputStream**
    - **PipedReader**
    - **PipedWriter**

7.1 PipedInputStream & PipedOutputStream

- **Pipes** in IO provide a link between two threads running in the JVM at the same time. So, Pipes are used both as a source and a destination.
- PipedInputStream is also piped with PipedOutputStream.
- So, data can be written using PipedOutputStream and can be read using PipedInputStream.
- But using both threads at the same time will create a deadlock for the threads.
- A pipe is said to be broken if a thread that was providing data bytes to the connected piped output stream is no longer alive.
- The Java.io.PipedInputStream class and the Java.io.PipedOutputStream class provide a convenient means to move input from one thread output for another thread.



Example13:using PipedInputstream &PipedOutputStream

```
import java.io.*;

class PipedByteDemo {

   public static void main(String[] args) throws IOException {

      // Create piped streams
```

```
        PipedOutputStream pos = new PipedOutputStream();
        PipedInputStream pis = new PipedInputStream(pos); // connect
streams
        // Writing data (byte format)
        String str = "Hello Navjot";
        pos.write(str.getBytes());
        pos.close(); // close after writing
        // Reading data
        int data;
        System.out.print("Output: ");
        while ((data = pis.read()) != -1) {
            System.out.print((char) data);
        }
        pis.close();
    }
}
```

Output: Hello Navjot

PipedWriter Class

- **The PipedWriter class is used to write java pipe as a stream of characters**.
- This class is used generally for **writing text**. Generally **PipedWriter is connected to a PipedReader** and used by different threads.
- **Constructor:**

Constructor	Description
PipedWriter()	It creates a piped writer that is not yet connected to a piped reader.
PipedWriter(PipedReader snk)	It creates a piped writer connected to the specified piped reader.

PipedWriter Methods:

Modifier and Type	Method	Method
void	close()	It closes this piped output stream and releases any system resources associated with this stream.
void	connect(PipedReader snk)	It connects this piped writer to a receiver.
void	flush()	It flushes this output stream and forces any buffered output characters to be written out.
void	write(char[] cbuf, int off, int len)	It writes len characters from the specified character array starting at offset off to this piped output stream.
void	write(int c)	It writes the specified char to the piped output stream.

PipedReader Class

- The **PipedReader class is used to read the contents of a pipe as a stream** of characters. This class is used generally to read text.
- PipedReader class **must be connected to the same PipedWriter** and are used by different threads.

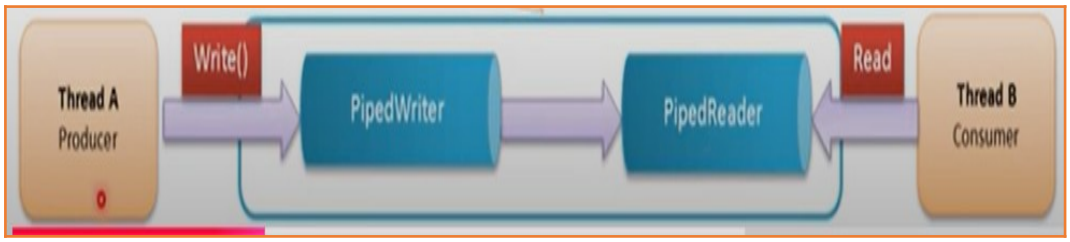
- **Constructor:**

Constructor	Description
PipedReader(int pipeSize)	It creates a PipedReader so that it is not yet connected and uses the specified pipe size for the pipe's buffer.
PipedReader(PipedWriter src)	It creates a PipedReader so that it is connected to the piped writer src.
PipedReader(PipedWriter src, int pipeSize)	It creates a PipedReader so that it is connected to the piped writer src and uses the specified pipe size for the pipe's buffer.
PipedReader()	It creates a PipedReader so that it is not yet connected.

PipedReader class Methods

Method

Modifier and Type	Method	Method
void	close()	It closes this piped stream and releases any system resources associated with the stream.
void	connect(PipedWriter src)	It causes this piped reader to be connected to the piped writer src.
int	read()	It reads the next character of data from this piped stream.
int	read(char[] cbuf, int off, int len)	It reads up to len characters of data from this piped stream into an array of characters.
boolean	ready()	It tells whether this stream is ready to be read.



Example14: using PiperWriter & PipedReader Class

```
import java.io.*;
class PipedDemo {
    public static void main(String[] args) throws IOException {
        // Create piped writer and reader
        PipedWriter pw = new PipedWriter();
        PipedReader pr = new PipedReader(pw); // connect both
        // Writing data
        String str = "Hello Navjot";
        pw.write(str);
        pw.close(); // important to close after writing
        // Reading data
        int ch;
        System.out.print("Output: ");
        while ((ch = pr.read()) != -1) {
            System.out.print((char) ch);
        }
```

```
        pr.close();
    }
}
```

Output: Hello Navjot

Writer Class

- The java.io.Writer class is a superclass of all character input stream classes.
- It declares methods to write characters to the stream ,closing the stream, flushing stream etc.
- It implements the **Closeable, Flushable, and Appendable interfaces**.
- Writer class methods are shown in the given table

Method	Description
Writer append(char *ch*)	Appends *ch* to the end of the invoking output stream. Returns a reference to the invoking stream.
Writer append(CharSequence *chars*)	Appends *chars* to the end of the invoking output stream. Returns a reference to the invoking stream. **CharSequence** is an interface that defines read-only operations on a sequence of characters.
Writer append(CharSequence *chars*, int *begin*, int *end*)	Appends the sequence of *chars* starting at *begin* and stopping with *end* to the end of the invoking output stream. Returns a reference to the invoking stream. **CharSequence** is an interface that defines read-only operations on a sequence of characters.
abstract void close( )	Closes the output stream. Further write attempts will generate an **IOException**.
abstract void flush( )	Causes any output that has been buffered to be sent to its destination. That is, it flushes the output buffer.
void write(int *ch*)	Writes a single character to the invoking output stream. Note that the parameter is an **int**, which allows you to call **write( )** with expressions without having to cast them back to **char**.
void write(char *buffer*[ ])	Writes a complete array of characters to the invoking output stream.
abstract void write(char *buffer*[ ], int *offset*, int *numChars*)	Writes a subrange of *numChars* characters from the array *buffer*, beginning at *buffer*[*offset*] to the invoking output stream.
void write(String *str*)	Writes *str* to the invoking output stream.
void write(String *str*, int *offset*, int *numChars*)	Writes a subrange of *numChars* characters from the array *str*, beginning at the specified *offset*.

FileWriter Class

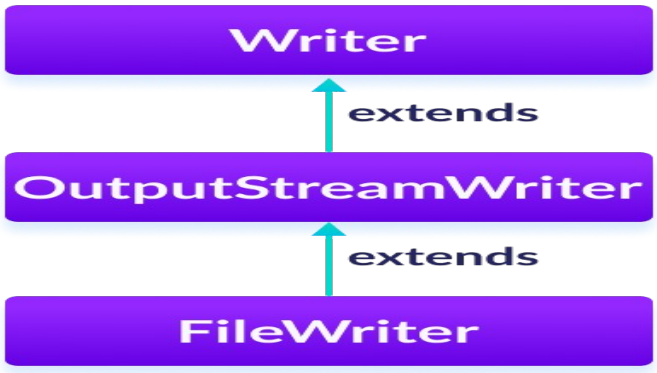
- FileWriter is meant for writing streams of characters. For writing streams of raw bytes, consider using a FileOutputStream.
- FileWriter together with its write() method to create and write some text into a file.
- **Create a FileWriter**

1. **Using the name of the file**

```
FileWriter output = new FileWriter(String name);
```

2. Using an object of the file

FileWriter input = new FileWriter(File fileObj);



FileWriter Class Methods

Method	Description
void write(String text)	It is used to write the string into FileWriter.
void write(char c)	It is used to write the char into FileWriter.
void write(char[] c)	It is used to write char array into FileWriter.
void flush()	It is used to flushes the data of FileWriter.
void close()	It is used to close the FileWriter.

Example15: Write into a file using the FileWriter Class

```
import java.io.FileWriter;
public class FileWriterExample {
   public static void main(String args[]) {
      try {
         // Create FileWriter object
         FileWriter fw = new FileWriter("D:\\testwrite.txt");
         // Write string into file
         fw.write("Hello Students! This is Java Class.");
         // Close the file
         fw.close();
```

```
            System.out.println("Data written successfully...");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT:

Data written successfully...

Reader Class

- The java.io.Reader class is a superclass of all character input stream classes.
- It declares methods to read one or more characters, marking the position in a stream and resetting back to that position, closing the stream, skipping characters over input, etc.
- It implements the **Closeable and Readable interfaces**.
- Reader class methods are shown in the given table

Method	Description
abstract void close( )	Closes the input source. Further read attempts will generate an **IOException**.
void mark(int *numChars*)	Places a mark at the current point in the input stream that will remain valid until *numChars* characters are read.
boolean markSupported( )	Returns **true** if **mark( )/reset( )** are supported on this stream.
int read( )	Returns an integer representation of the next available character from the invoking input stream. –1 is returned when the end of the file is encountered.
int read(char *buffer*[ ])	Attempts to read up to *buffer.length* characters into *buffer* and returns the actual number of characters that were successfully read. –1 is returned when the end of the file is encountered.
int read(CharBuffer *buffer*)	Attempts to read characters into *buffer* and returns the actual number of characters that were successfully read. –1 is returned when the end of the file is encountered.
abstract int read(char *buffer*[ ], int *offset*, int *numChars*)	Attempts to read up to *numChars* characters into *buffer* starting at *buffer*[*offset*], returning the number of characters successfully read. –1 is returned when the end of the file is encountered.
boolean ready( )	Returns **true** if the next input request will not wait. Otherwise, it returns **false**.
void reset( )	Resets the input pointer to the previously set mark.
long skip(long *numChars*)	Skips over *numChars* characters of input, returning the number of characters actually skipped.

FileReader Class

- FileReader class inherits from the InputStreamReader class.

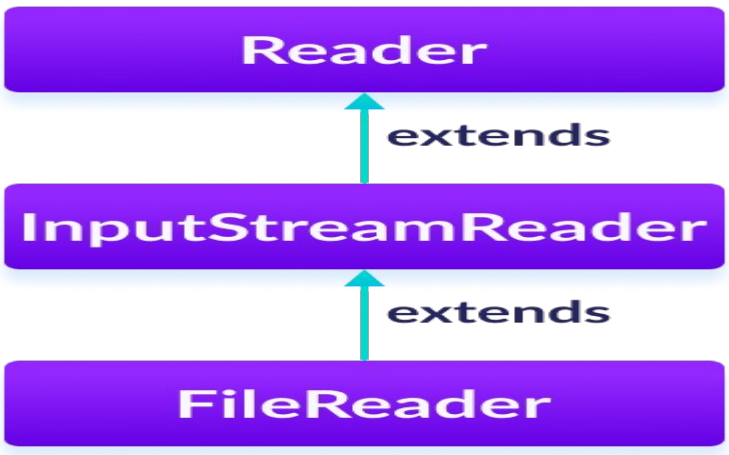
- FileReader is used for reading streams of characters.
- The FileReader class creates a **Reader** that you can use to read the contents of a file.
- To create a file reader, we must import the java.io.FileReader package first.
- Create a FileReader Class

1. **Using the name of the file**

   FileReader input = new FileReader(String name);

2. Using an object of the file

FileReader input = new FileReader(File fileObj);



FileReader Class Methods

Method	Description
int read()	It is used to return a character in ASCII form. It returns -1 at the end of file.
void close()	It is used to close the FileReader class.

Example16: Read from a file using the FileReader Class

import java.io.FileReader;

```
public class FileReaderExample {
   public static void main(String args[]) {
      try {
         // Create FileReader object
         FileReader fr = new FileReader("D:\\testwrite.txt");
         int i;
         // Read character by character
         while ((i = fr.read()) != -1) {
            System.out.print((char) i);
         }
         // Close the file
         fr.close();
      } catch (Exception e) {
         System.out.println(e);
      }
   }
}
```

Output:

Hello Students! This is a Java Class

Example4: Read from a console using FileReader Class

```
import java.io.*;
public class ReadingDemo {
   public static void main(String[] args) throws IOException {
      BufferedInputStream read = new BufferedInputStream(System.in);
      try {
         System.out.print("Enter any character: ");
```

```
        char c = (char) read.read();
        System.out.println("You have entered " + c);
    }
    catch (Exception e) {
        System.out.println(e);
    }
    finally {
        read.close();
    }
}}
```

Output:

Enter any character: h

You have entered h

Example17: Sample Program on Reading/Writing Files using File IO streams

```
import java.io.*;
public class FileIOStreamDemo {
   public static void main(String []args)
   {
      int ch;
      try{
         FileOutputStream fileoutput = new FileOutputStream("C:\\Users\\
91842\\JAVA_PROGRAM\\UNIT 2\\mydata.txt");
         for(int i =1;i<=10;i++)
            fileoutput.write(i);
            fileoutput.close();
         }
```

```
        catch(IOException e)
        {
            e.printStackTrace();
        }
try
        {
            FileInputStream fileinput = new FileInputStream("C:\\Users\\91842\\
JAVA_PROGRAM\\UNIT 2\\mydata.txt");
            while((ch=fileinput.read())!=-1)
                System.out.println(ch);
            fileinput.close();
        }
        catch(IOException e)
        {
            e.printStackTrace();
        }
    }
}
```

Output:

```
PS C:\Users\91842\JAVA_PROGRAM\UNIT 2> javac FileIOStreamDemo.java
PS C:\Users\91842\JAVA_PROGRAM\UNIT 2> java FileIOStreamDemo
1
2
3
4
5
6
```

7

8

9

10

Explanation:

- In this program in the main() method of FileIOStreamDemo, an instance fileoutput of FileOutputStream is created to write to the file “mydata.txt”.
- Then using for loop , we write 1 to 10 stored in the index variable “I” using the write() method to the file “mydata.txt”. The output stream is then closed by calling the close() method.
- To read bytes from the file “mydata.txt”, we create an instance fileinput of the FileInputStream Class.
- Then read the bytes from “mydata.txt” file one by one and display it on the screen and assign it to variable “ch” and check whteher it is -1 or not.
- If ch!= -1 the byte is displayed on the screen and if ch==-1 then it indicates that endoffile (EOF) is reached and the loop terminates.